

# Proxmox 3 - Práctico

WORKSHOP

**“Tu vida pasa por la consola”**



# Proxmox

---



**X**PRO**X**MO**X**

# ¿Qué es KVM?

---



**KVM** (Kernel-based Virtual Machine) permite ejecutar máquinas virtuales completas que emulan hardware a los recursos del servidor físico (CPU, RAM, almacenamiento). Cada VM es independiente y puede ejecutar su propio sistema operativo (Linux, Windows, BSD, etc).



# Virtualización Completa (KVM)



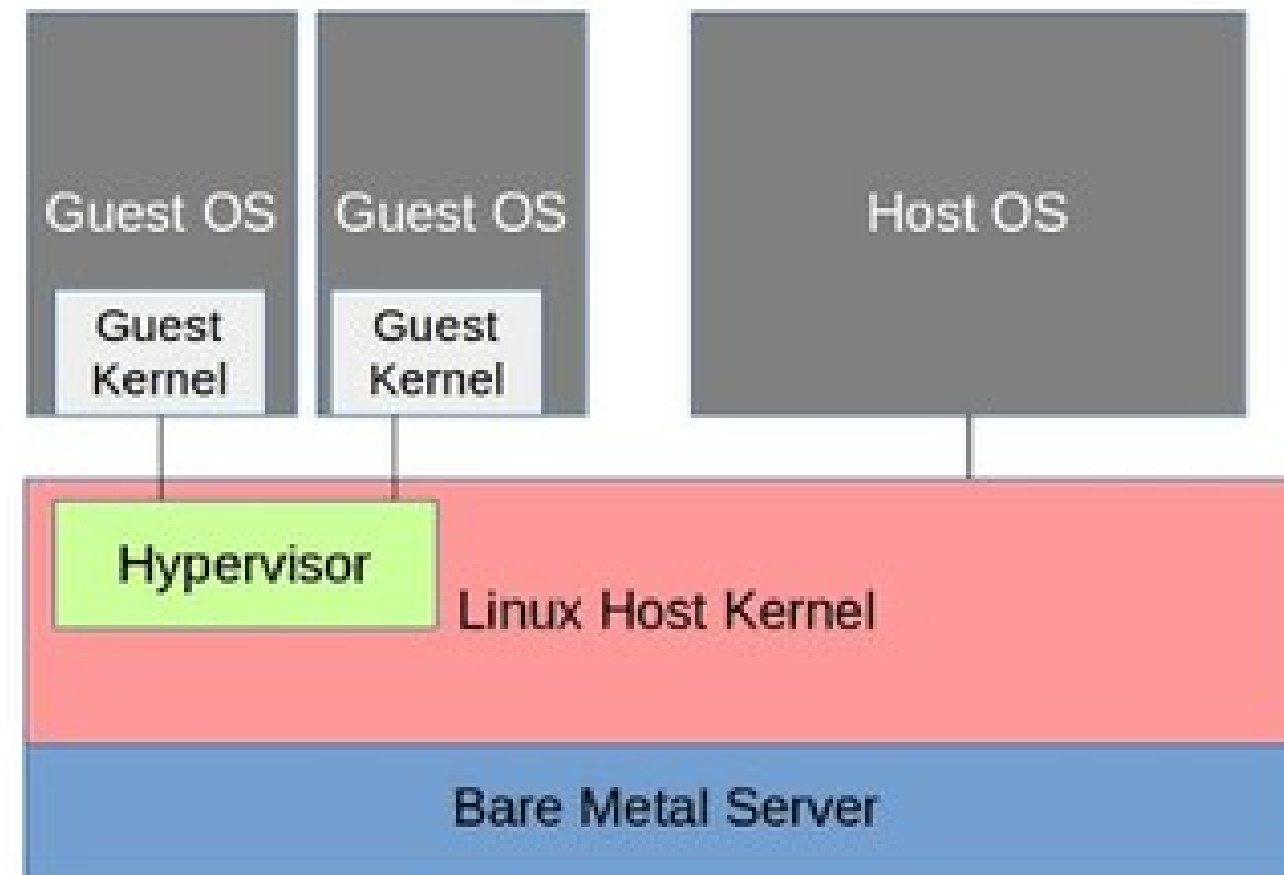
- KVM convierte a Linux en un hipervisor tipo 1.
- Permite ejecutar sistemas operativos completos (Windows, GNU/Linux, BSD) como máquinas virtuales.
- Ventaja: Máxima compatibilidad y aislamiento.
- Desventaja: Mayor consumo de recursos en comparación con los contenedores.





# KVM Hypervisor

## Architecture



# Requisitos para Crear una VM



Antes de crear una VM, hay que asegurarse de:

- Haber subido una ISO de instalación al almacenamiento.
- Tener suficiente espacio y recursos libres (CPU/RAM).



# Requisitos para Crear una VM



Para subir una ISO desde el panel web:

- Ir a **Datacenter** → **Node** → **Local (Storage)**.
- Hacer click en **Content** → **Upload**.
- Seleccionar el tipo **ISO Image** y subir el archivo.

The screenshot displays the Proxmox VE web interface. On the left, the 'Server View' sidebar shows the 'Datacenter' tree with 'pve02' selected. Under 'pve02', several VMs are listed, and at the bottom, 'local (pve02)' is highlighted. The main panel shows 'Storage 'local' on node 'pve02''. The left sidebar of this panel lists 'Summary', 'Backups', 'VM Disks', 'CT Volumes', 'ISO Images' (selected), 'CT Templates', 'Snippets', and 'Permissions'. The right side of this panel shows a table of existing ISOs: 'FreeBSD-14.3-RELEASE-amd64-dvd1.iso', 'slackware64-15.0-install-dvd.iso', 'TinyCore-current.iso', and 'TinyCorePure64-16.2.iso'. Above the table are buttons for 'Upload', 'Download from URL', and 'Remove'. An 'Upload' modal window is open on the right, showing the file path 'C:\fakepath\slackware64-live-15', the filename 'slackware64-live-15.0.iso', a size of '3.91 GiB', and a MIME type of 'application/vnd.efi.iso'. The modal also includes a 'Checksum' field and a warning message: 'Uploads are stored temporarily in '/var/tmp/', make sure there is enough free space.' At the bottom of the modal are 'Abort' and 'Upload' buttons.



# Crear una Máquina Virtual (General y OS)



- Acceder al nodo y hacer clic en **Create VM**.

 Create VM

Create: Virtual Machine

General OS System Disks CPU Memory Network Confirm

Node: pve02 Resource Pool:   
VM ID: 107   
Name: Slackware64

Help Advanced ☐ Back Next

Create: Virtual Machine

General OS System Disks CPU Memory Network Confirm

☒ Use CD/DVD disc image file (iso)   
Storage: local   
ISO image: slackware64-15.0-inst   
Guest OS:   
Type: Linux   
Version: 6.x - 2.6 Kernel   
☐ Use physical CD/DVD Drive   
☐ Do not use any media

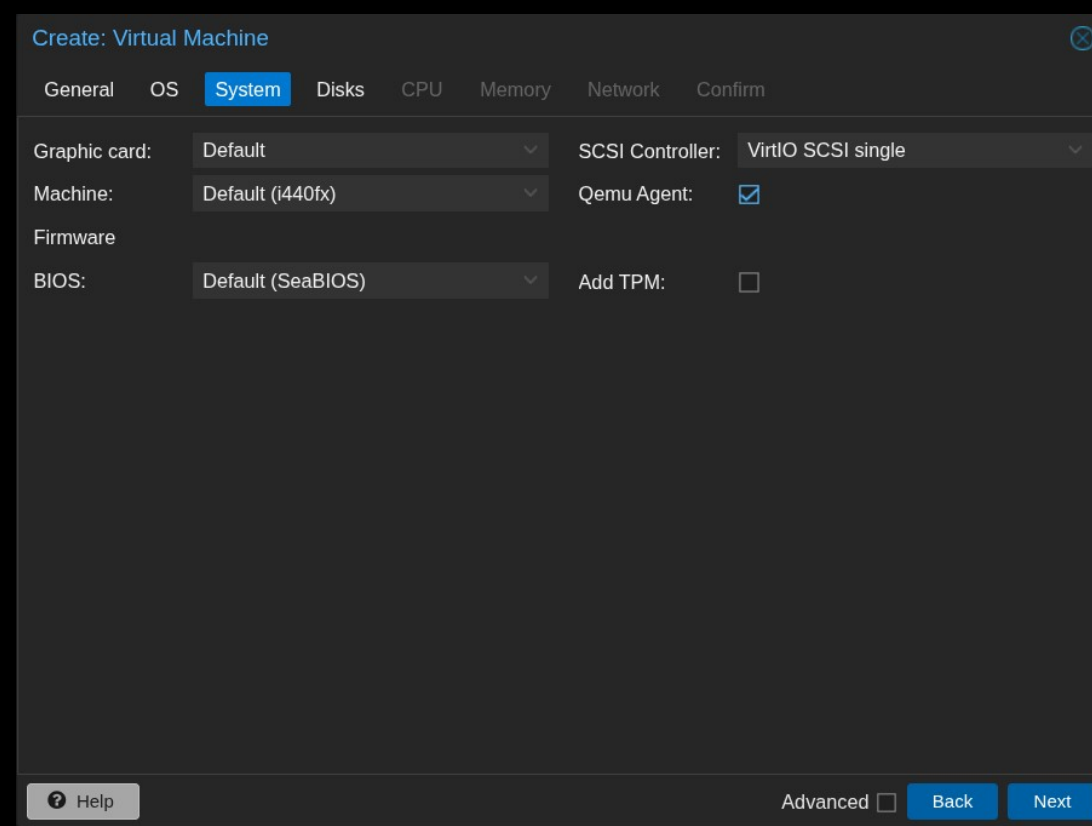
Advanced ☐ Back Next



# Crear una Máquina Virtual (System)



- **BIOS:** OVMF (UEFI) o SeaBIOS.
  - Para Windows 11, usa UEFI + TPM.
  - OVMF (si tenemos previsto utilizar PCIe passthrough).
- **Machine:** q35 o i440fx.
  - q35 es más moderno.
- **TPM:** Opcional. Requerido para Windows 11.



# Crear una Máquina Virtual (Disks)



- **Bus/Device:** VirtIO (mejor rendimiento SCSI o Blcock).
  - Se recomienda **SCSI VirtIO SCSI single**, cada disco tendrá su propio controlador **VirtIO SCSI**, y **QEMU**.
- **Disk size:** Define el tamaño en GB.
- **Storage:** El tipo de almacenamiento (local-lvm, Ceph, etc).
- **Cache:** Write back (rápido) o Direct sync (seguro).



# Crear una Máquina Virtual (Disks)



Create: Virtual Machine

General OS System **Disks** CPU Memory Network Confirm

scsi0

**Disk** Bandwidth

Bus/Device: SCSI 0 Cache: Write back

SCSI Controller: VirtIO SCSI single Discard: ☐

Storage: local IO thread: ☒

Disk size (GiB): 32

Format: QEMU image format

Add

Import

Help

Advanced ☐ Back Next



# Crear una Máquina Virtual (CPU)



- **Sockets y cores:** Define cuántos vCPUs asignarás.
  - CPU con 4 núcleos o dos zócalos con dos núcleos cada uno es prácticamente irrelevante.
  - Si no se conoce la carga de trabajo de la máquina virtual, lo más recomendable suele ser establecer el número **total de núcleos** en 2.

# lscpu



# Crear una Máquina Virtual (CPU)



- **Type:** Host o kvm64.
  - (Host para máximo rendimiento).
    - kvm64 (x86-64-v1) : Compatible con CPU Intel  $\geq$  Pentium 4, CPU AMD  $\geq$  Phenom.
    - x86-64-v2 : Compatible con CPU Intel  $\geq$  Nehalem, CPU AMD  $\geq$  Opteron G3. Se agregaron indicadores de CPU en comparación con x86-64-v1 : +cx16 , +lahf-lm , +popcnt , +pni , +sse4.1 , +sse4.2 , +ssse3 .
    - x86-64-v2-AES : Compatible con CPU Intel  $\geq$  Westmere, CPU AMD  $\geq$  Opteron G4. Se agregaron indicadores de CPU en comparación con x86-64-v2 : +aes .
    - x86-64-v3 : Compatible con CPU Intel  $\geq$  Haswell, CPU AMD  $\geq$  EPYC. Se agregaron indicadores de CPU en comparación con x86-64-v2-AES : +avx , +avx2 , +bmi1 , +bmi2 , +f16c , +fma , +movbe , +xsave .
    - x86-64-v4 : Compatible con CPU Intel  $\geq$  Skylake, CPU AMD  $\geq$  EPYC v4 Genoa. Se agregaron indicadores de CPU en comparación con x86-64-v3 : +avx512f , +avx512bw , +avx512cd , +avx512dq , +avx512vl.



# Crear una Máquina Virtual (CPU)



Create: Virtual Machine

General OS System Disks CPU Memory Network Confirm

Sockets: 2 Type: host

Cores: 2 Total cores: 4

? Help Advanced ☐ Back Next



# Crear una Máquina Virtual (Memory)



- **RAM:** Por ejemplo 2048 MB (2 GB).
- **Ballooning:** Activado si quieres memoria dinámica.

Create: Virtual Machine

General OS System Disks CPU **Memory** Network Confirm

Memory (MiB): 4096

Minimum memory (MiB): 4096

Shares: Default (1000)

Ballooning Device: ☒

? Help Advanced ☒ Back Next



# Crear una Máquina Virtual (Network)



- **Bridge:** vubr0 (por defecto).
- **Model:** VirtIO (paravirtualizado).

Create: Virtual Machine

General OS System Disks CPU Memory **Network** Confirm

☐ No network device

Bridge: vubr0 Model: VirtIO (paravirtualized)

VLAN Tag: no VLAN MAC address: auto

Firewall: ☒

---

Disconnect: ☐ Rate limit (MB/s): unlimited

MTU: Same as bridge Multiqueue:

[? Help](#) Advanced ☒ [Back](#) [Next](#)



# Crear una Máquina Virtual (Confirm)



Create: Virtual Machine

General

OS

System

Disks

CPU

Memory

Network

Confirm

| Key ↑    | Value                                                  |
|----------|--------------------------------------------------------|
| agent    | 1                                                      |
| cores    | 2                                                      |
| cpu      | host                                                   |
| ide2     | local:iso/slackware64-15.0-install-dvd.iso,media=cdrom |
| memory   | 4096                                                   |
| name     | Slackware64                                            |
| net0     | virtio,bridge=vmbr0,firewall=1                         |
| nodename | pve02                                                  |
| numa     | 0                                                      |
| ostype   | l26                                                    |
| scsi0    | local:32,format=qcow2,iothread=on,cache=writeback      |
| scsihw   | virtio-scsi-single                                     |
| sockets  | 2                                                      |
| vmid     | 107                                                    |

☒ Start after created

Advanced ☒

Back

Finish



# Instalación de Drivers VirtIO



- Si instalamos Windows, deberemos montar la ISO de drivers VirtIO para:
  - Disco (VirtIO SCSI).
  - Red (VirtIO Net).
- Pasos:
  - Bajar: <https://fedorapeople.org/groups/virt/virtio-win/direct-downloads/stable-virtio/virtio-win.iso>
  - Subir la ISO virtio-win.iso
  - Montarlo como CD/DVD Drive 2.
  - Durante la instalación de Windows, cargar drivers desde ese CD.





# ! Vamos a Practicar !



# Comandos para la VM

---



- Listas las VM.

# qm list

- Crear Snapshot

# qm snapshot <VMID> <snapshot-name> --description "Comentario"

- Ejemplo:

# qm snapshot 108 pre-update-alpine --description "Antes de actualizar"

- Listar snapshot

# qm listsnapshot 108





# ! Vamos a Practicar !



# Comandos para la VM

---



- Para revertir:

```
# qm rollback 108 pre-update-alpine
```

- Borrar snapshot

```
# qm delsnapshot 108 pre-update-alpine
```

- Clonación de Máquinas Virtuales

```
# qm clone <VMID_ORIGINAL> <VMID_CLONADO> --name NUEVO_NOMBRE
```

```
# qm clone 108 109 --name alpinelinux-2
```



# Comandos para la VM

---



- Backups

```
# vzdump <VMID> --storage local --mode snapshot
```

```
# vzdump 108 --storage local --mode snapshot
```

- Restaurar Backups

```
# qmrestore /var/lib/vz/dump/backup.tar <VMID_NUEVO>
```

```
# qmrestore /var/lib/vz/dump/vzdump-qemu-108-2025_11_02-23_10_24.vma 108
```

- Migración de Vms en Cluster

```
# qm migrate <VIMD> <NODO_DESTINO> --online
```

```
# qm migrate 108 pve-node2 --online
```



! Vamos a Practicar !



# Comandos para la VM

---



- Parar

# qm stop 108

- Levantar

# qm start 108

- Reiniciar

# qm reset 108

- Estado

# qm status 108



# Comandos para la VM

---



- Apagar

# qm shutdown 108

- Eliminar

# qm destroy 108



# Buenas Prácticas

---



- Usa **VirtIO** siempre que sea posible para mejorar el rendimiento.
- Asigna **CPU/RAM** según el uso real, evitando el sobreaprovisionamiento.
- Mantén snapshots solo cuando sean necesarios.
- Implementa backups programados.
- Si usas **Windows Vms**, instala siempre los drivers **qemu-guest-agent**.





# ! Vamos a Practicar !



# ¿Qué es LXC en Proxmox VE?



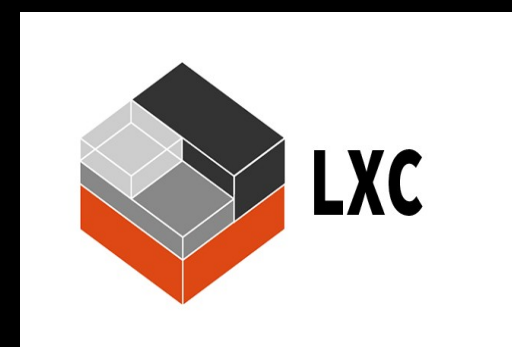
- LXC (Linux Containers) es una tecnología de virtualización a nivel de sistema operativo.
- Comparte el kernel del host, por lo que es más eficiente que las Vms.
- Ideal para ejecutar entornos Linux ligeros y escalables.

## **Ventajas**

- Menor uso de recursos (RAM y CPU).
- Arranque casi instantáneo.
- Fácil de clonar y replicar.

## **Limitaciones**

- Solo soporta sistemas GNU/Linux.
- Aislamiento menor que una VM (aunque seguro si se configura bien).
- No puede usar kernels distintos al de host.

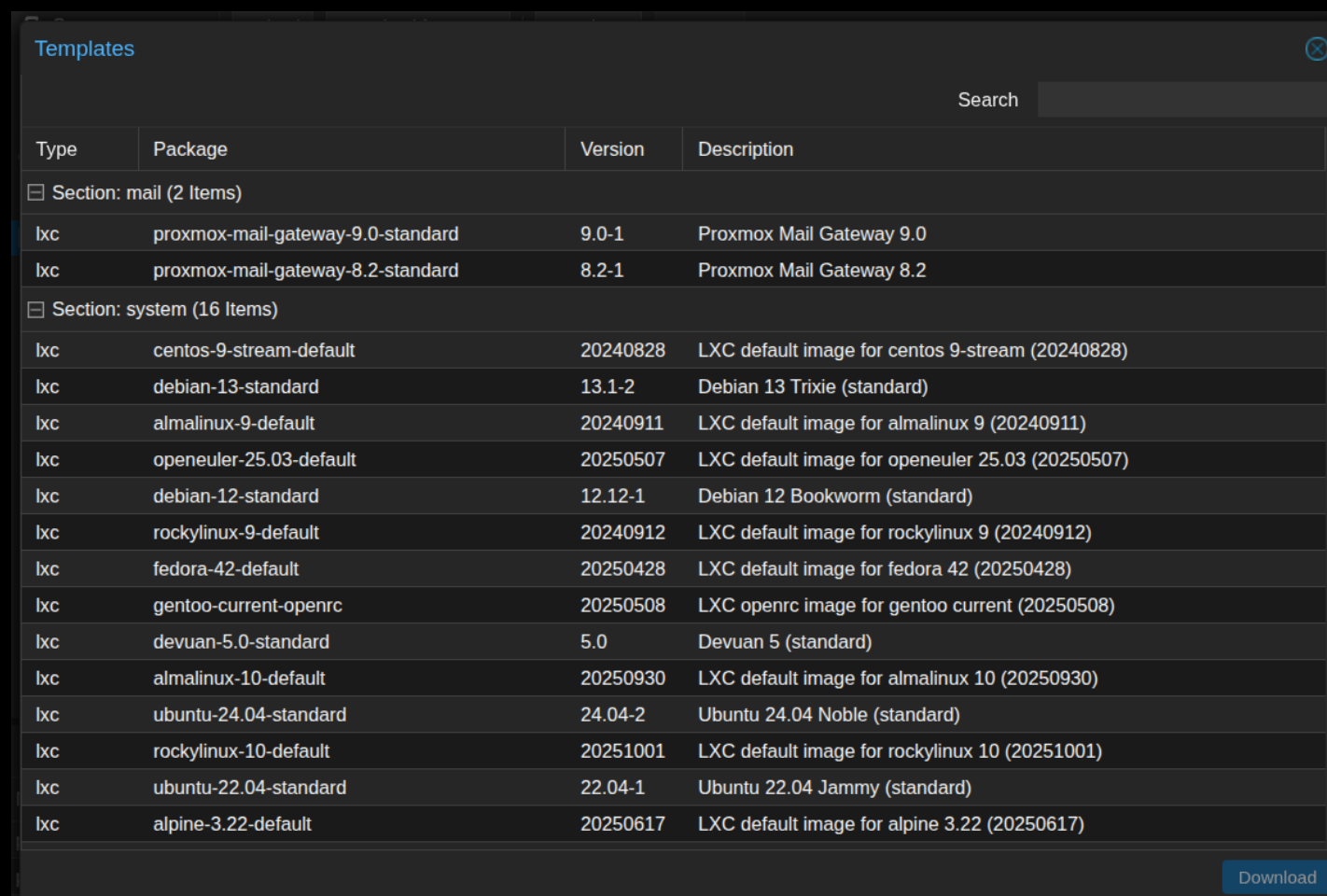
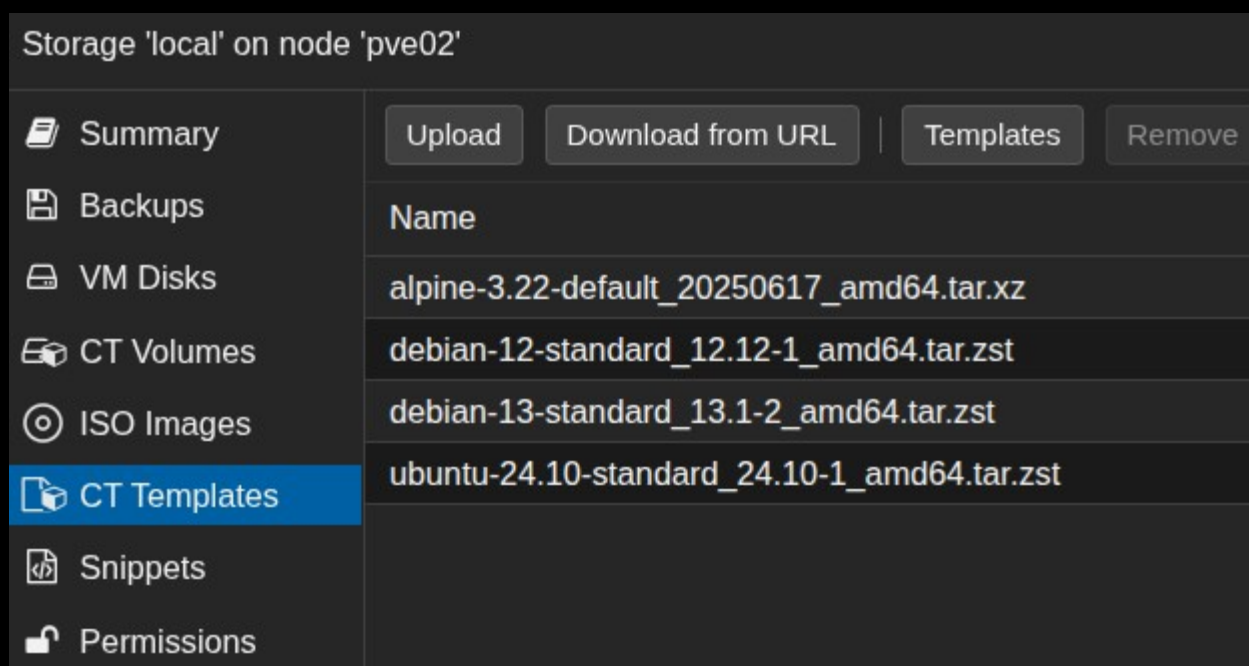


# Descargar Plantilla de Contenedores



Proxmox usa plantillas preconstruidas para crear contendores.

- Ir al Datacenter → Node → Local (storage) → CT Template
- Hacer clic en Templates.
- Elegir la plantilla que se desea (Debian, Ubuntu, Alpine, CentoOS, etc).
- Descargarla





# ! Vamos a Practicar !



# Descargar Plantilla de Contenedores



Desde el CLI:

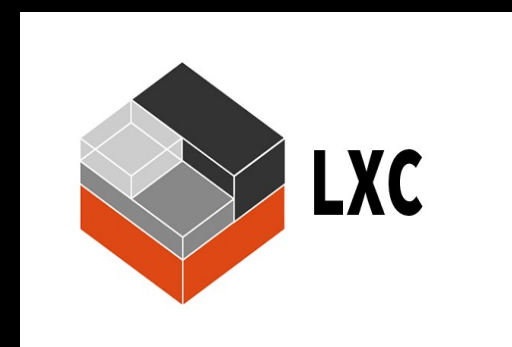
```
# pveam update  
# pveam available  
# pveam download local debian-13-standard_13.1-2_amd64.tar.zst
```

Listar

```
# pveam list local
```

Remover

```
# pveam remove local:vztmpl/debian-12-standard_12.12-1_amd64.tar.zst
```





# ! Vamos a Practicar !



# Crear un Contenedor LXC desde la Web



- Haz clic en **Create CT**.

 Create CT

Create: LXC Container

General Template Disks CPU Memory Network DNS Confirm

Node: pve02 Resource Pool:

CT ID: 109 Password: .....

Hostname: Debian13 Confirm password: .....

Unprivileged container: ☒

Nesting: ☒

SSH public key(s):

Load SSH Key File

Tags

Help Advanced ☒ Back Next

Create: LXC Container

General Template Disks CPU Memory Network DNS Confirm

Storage: local

Template: debian-13-standard\_13.1-2\_amd64

Help Advanced ☒ Back Next



# Crear un Contenedor LXC desde la Web



- Haz clic en **Create CT**.

 Create CT

Create: LXC Container

General Template **Disks** CPU Memory Network DNS Confirm

rootfs 

Storage: local

Disk size (GiB): 8

Enable quota: ☐ ACLs: Default

Mount options:  Skip replication: ☐

 Add

 Help Advanced ☒ Back Next

Create: LXC Container

General Template Disks **CPU** Memory Network DNS Confirm

Cores: 2

CPU limit: unlimited CPU units: 100

 Help Advanced ☒ Back Next



# Crear un Contenedor LXC desde la Web



- Haz clic en **Create CT**.

 Create CT

Create: LXC Container

General Template Disks CPU **Memory** Network DNS Confirm

Memory (MiB): 2048

Swap (MiB): 2048

Help Advanced ☒ Back Next

Create: LXC Container

General Template Disks CPU Memory **Network** DNS Confirm

Name: eth0 IPv4: ☐ Static ☒ DHCP

MAC address: auto IPv4/CIDR:

Bridge: vmbri0 Gateway (IPv4):

VLAN Tag: no VLAN IPv6: ☒ Static ☐ DHCP ☐ SLAAC

Firewall: ☒ IPv6/CIDR: None

Disconnect: ☐ Rate limit (MB/s): unlimited

MTU: Same as bridge

Help Advanced ☒ Back Next



# Crear un Contenedor LXC desde la Web



- Haz clic en **Create CT**.

 Create CT

Create: LXC Container

General Template Disks CPU Memory Network **DNS** Confirm

DNS domain: use host settings

DNS servers: 8.8.8.8

Advanced ☒ Back Next

Create: LXC Container

General Template Disks CPU Memory Network DNS **Confirm**

| Key ↑           | Value                                                |
|-----------------|------------------------------------------------------|
| cores           | 2                                                    |
| features        | nesting=1                                            |
| hostname        | Debian13                                             |
| memory          | 2048                                                 |
| nameserver      | 8.8.8.8                                              |
| net0            | name=eth0,bridge=vmbr0,firewall=1,ip=dhcp            |
| nodename        | pve02                                                |
| ostemplate      | local:vztmpl/debian-13-standard_13.1-2_amd64.tar.zst |
| pool            |                                                      |
| rootfs          | local:8                                              |
| ssh-public-keys |                                                      |
| swap            | 2048                                                 |
| unprivileged    | 1                                                    |
| vmid            | 109                                                  |

☒ Start after created

Advanced ☒ Back Finish





# ! Vamos a Practicar !



# Comandos Básicos de Gestión de Contenedores (CLI)



- Listar contenedores

# pct list

- Iniciar / Deter

# pct start <CTID>

# pct stop <CTID>

- Reiniciar

# pct reboot <CTID>

- Ver estado

# pct status <CTID>



# Comandos Básicos de Gestión de Contenedores (CLI)



- Acceder al shell (sin login)

# pct enter <CTID>

- Acceder a la consola (con login)

# pct console <CTID>

- Eliminar contenedor

# pct destroy <CTID>

- Ver configuración

# pct config <CTID>



# Comandos Básicos de Gestión de Contenedores (CLI)



- Puedes ajustar los recursos dinámicamente:

```
# pct set <CTID> -memory 4096
```

```
# pct set <CTID> -cores 2
```

- Ver espacio de disco

```
# pct df <CTID>
```





# ! Vamos a Practicar !





# Snapshots en Contenedores (CLI)

Se puede realizar snapshot si utilizamos almacenamiento (ZFS, Ceph, etc).

- Crear snapshot

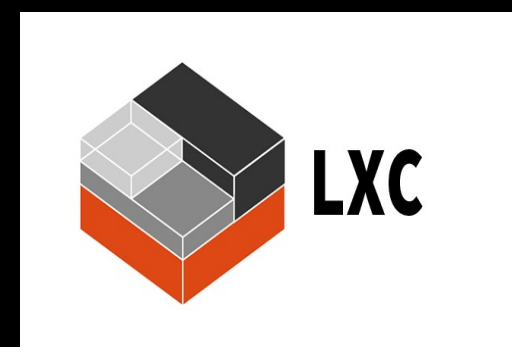
# pct snapshot <CTID> nombre-snapshot

- Revertir snapshot

# pct rollback <CTID> nombre-snapshot

- Listar snapshots

# pct listsnapshot <CTID>





# ! Vamos a Practicar !



# Clonar un Contenedor LXC (CLI)

---



- Clonar

```
# pct clone <CTID_ORIGINAL> <CTID_CLON> --hostname nuevo-host
```

```
# pct clone 100
```





# ! Vamos a Practicar !





# Backup y Restauración de Contenedores LXC (CLI)

- Backup

```
# vzdump <CTID> --storage local --mode snapshot
```

```
# vzdump 100 --storage local --mode snapshot
```

- Restauración

```
# pct restore <CTID_NUEVO> /ruta/backup.tar.zst
```

```
# pct restore 200 /var/lib/vz/dump/vzdump-lxc-109-2025_11_03-08_56_02.tar
```





# ! Vamos a Practicar !





# Recursos Dinámicos y Limitaciones LXC (CLI)

- Podemos ajustar los recursos dinámicamente.

```
# pct set <CTID> -memory 2048
```

```
# pct set <CTID> -cores 2
```

- Limitar I/O de disco

```
# pct set <CTID> -mp0 /mnt/data,mp=/mnt,quota=1
```



# Montar Carpetas del Host en un Contenedor LXC (CLI)



- Ejemplo de bind mount:

```
# pct set <CTID> -mp0 /host/folder,mp=/container/folder
```

```
# pct set 109 /mnt,mp=/mnt
```





# ! Vamos a Practicar !





# Buenas Prácticas en LXC

---

- Usar **Unprivileged Containers** siempre que sea posible.
- Controla el uso de recursos (CPU/RAM).
- Realiza backups programados.
- Limita el acceso al sistema de archivos del host.
- Usa almacenamiento compatible con snapshots para agilizar la administración.





# BONUS TRACK



# ProxMenu

---



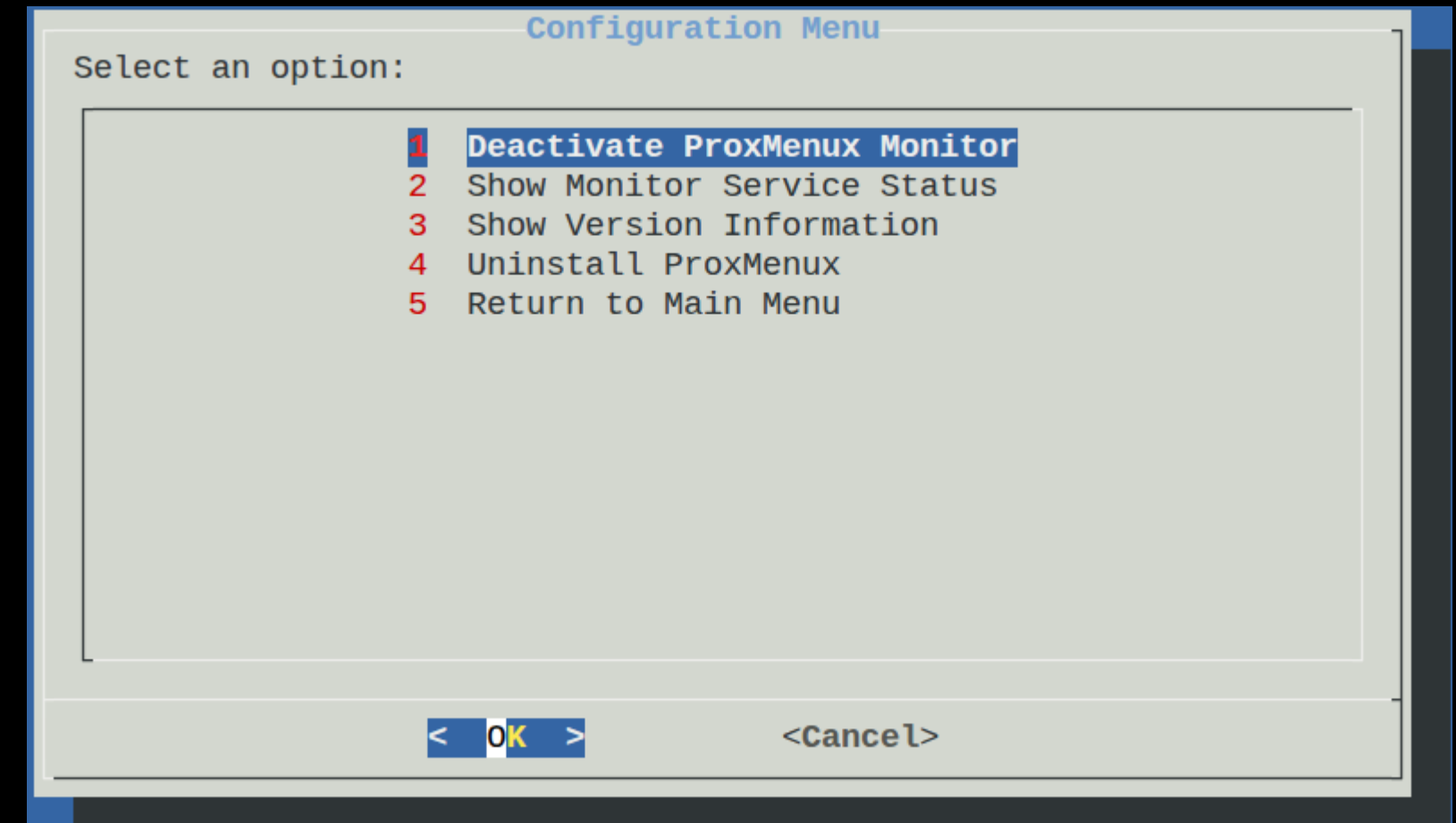
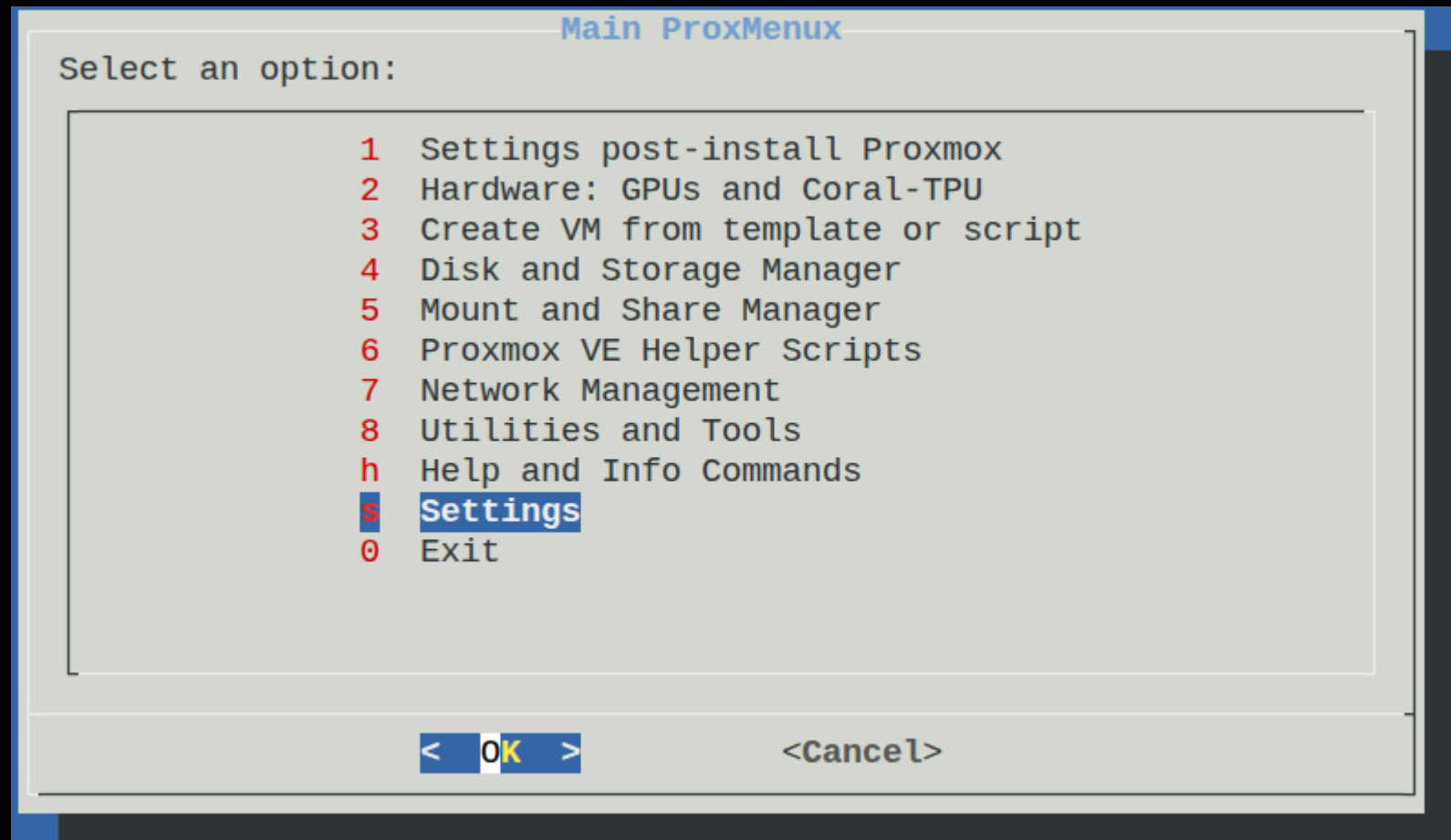
- Shell del pve
- GIT : <https://github.com/MacRimi/ProxMenux>

```
# bash -c "$(wget -qLO -  
https://raw.githubusercontent.com/MacRimi/ProxMenux/main/install_proxmenux.sh)"
```

# ProxMenu

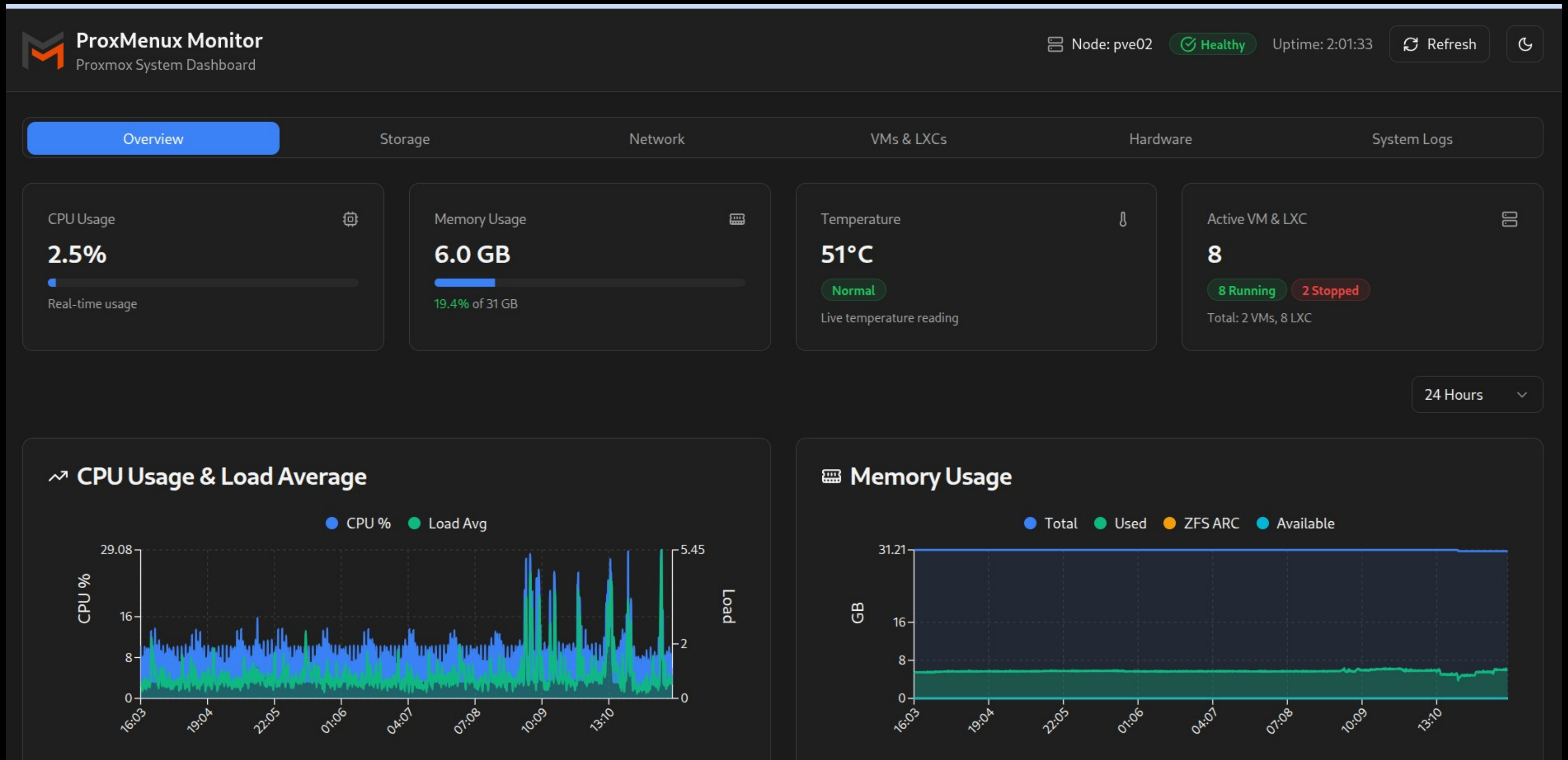


- Ejecutamos:  
# menu



# ProxMenu

http://IP:8008



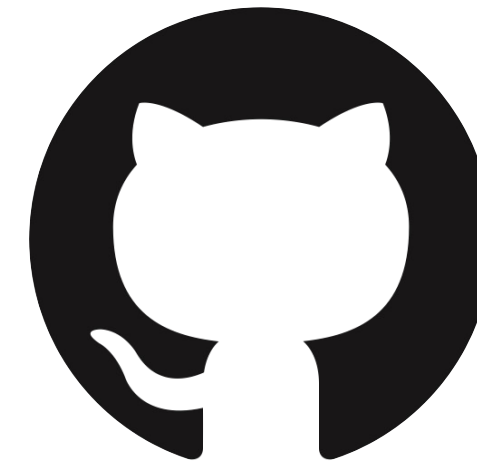
!!! Gracias Por



Estar !!!



Marcos Pablo Russo



[HTTPS://GITHUB.COM/MARCOSPR1974](https://github.com/MARCOSPR1974)



MARCOSPR1974@GMAIL.COM